ATELIER

OmniCore

Developer Guide

The complete architecture & codebase reference — every file, section, function, table, endpoint and webhook a developer needs to own this code.

Product	Multi-tenant SaaS omnichannel helpdesk & AI support platform
Repository	pnpm monorepo (artifacts + shared libs)
Runtime	Node.js 24 · TypeScript 5.9
Backend	Express 5 · Socket.io · PostgreSQL
Frontend	React 19 · Vite · Tailwind v4
Billing	Stripe + Paddle (dual-provider)

Generated 2026-07-06 · Confidential — internal engineering document

Contents

1 · Introduction & Mental Model

Atelier OmniCore is a multi-tenant SaaS omnichannel helpdesk. Businesses (tenants) sign up, create brands, invite support agents, and handle customer conversations from a real-time dashboard. Their customers (visitors) chat through an embeddable JavaScript widget, over email, or via API. An AI layer (Google Gemini) can auto-answer using a per-brand knowledge base.

Read this section first — once the four moving parts below click, every file in the repo has an obvious home.

The four runtime pieces

- API Server (artifacts/api-server) — Express 5 + Socket.io. The brain. Owns the database, auth, billing, AI, webhooks, and serves the embeddable widget. Written in TypeScript with CommonJS controllers.
- Dashboard (artifacts/dashboard) — React 19 + Vite SPA. What agents log into. Talks to the API over REST + Socket.io.
- Marketing Site (artifacts/marketing-site) — React + Vite with SSR/prerender. Public site: home, pricing, checkout, help, legal.
- Embeddable Widget — a vanilla-JS bundle the API server ships to any customer website. The floating chat bubble visitors use.

The core data hierarchy

Almost everything hangs off this chain. Memorise it and the database makes sense:

```
tenant  !’  brand  !’  visitor  !’  conversation  !’  message
   %          %
   %          % % %  knowledge_articles (AI answers)
   % % %  agents (support staff, scoped to brands)
```

The golden rule of multi-tenancy
Nearly every table carries a tenant_id column. Every query is scoped by it. When you add a feature or a table, you MUST carry tenant_id through or you will leak one customer's data into another's. This is the single most important invariant in the codebase.

2 · Languages & Tech Stack — What Is Used Where

One repo, a few languages. Here is exactly which language governs which surface, so you always know what you are editing.

Area / File	Language	Framework / Tooling
api-server/src/**.ts	TypeScript	Express 5, bundled with esbuild !’ dist/index.mjs
api-server controllers (.js)	JavaScript (CJS)	require()-based; interop shim at bundle time
api-server/schema.sql	SQL	PostgreSQL DDL (applied manually)
dashboard/src/App.tsx	TypeScript + JSX	React 19, Tailwind v4, Socket.io-client, Tiptap
dashboard AuthContext.jsx	JavaScript + JSX	Kept as JSX (allowJs: true in tsconfig)

Area / File	Language	Framework / Tooling
marketing-site/src/**	TypeScript + JSX	React, Wouter router, Vite SSR, Radix UI
Embeddable widget	Vanilla JS (IIFE)	No build step — plain browser JS
Styling everywhere	CSS (Tailwind)	Utility classes; Tailwind v4

> **Why controllers are .js but the entrypoint is .ts**
> The API server entrypoint and infrastructure (app.ts, index.ts, routes/index.ts, middleware) are TypeScript. The individual controllers are CommonJS .js files loaded via require(). esbuild handles the CJS!"ESM interop at bundle time via a require shim in the banner. This is intentional — do not "fix" it by converting everything to ESM imports.

Shared workspace libraries (lib/)

- @workspace/db — database client / Drizzle helpers shared across artifacts.
- @workspace/api-zod — generated Zod schemas from the OpenAPI spec (contract-first).

Regenerate contract helpers with: pnpm --filter @workspace/api-spec run codegen

3 · Monorepo Structure & How to Run It

This is a pnpm workspace. Each deployable app is an "artifact"; shared code lives in lib/. Apps never import each other directly — shared logic must go through a lib.

```
atelier-omnicore/
% % %  artifacts/
%    % % %  api-server/         Express + Socket.io backend (the brain)
%    % % %  dashboard/          React agent dashboard (SPA)
%    % % %  marketing-site/     Public React site (SSR/prerender)
%    % % %  mockup-sandbox/     Component preview playground
% % %  lib/                   Shared libraries (@workspace/*)
% % %  scripts/               Utility scripts (@workspace/scripts)
% % %  docs/                  !• this guide lives here
% % %  pnpm-workspace.yaml    Package discovery + dependency catalog
% % %  replit.md              Project README & conventions
```

Running & operating

API server	pnpm --filter @workspace/api-server run dev (port 8080)
Dashboard	pnpm --filter @workspace/dashboard run dev (port 5174)
Marketing site	pnpm --filter @workspace/marketing-site run dev
Full typecheck	pnpm run typecheck (run before shipping)
Per-app typecheck	pnpm --filter @workspace/<name> run typecheck

> **Do not run pnpm dev at the repo root**
> Apps run via Replit workflows, which inject PORT and BASE_PATH env vars. The root has no dev script by design. To run or restart an app, use its workflow — not a root-level shell command. Verify apps with typecheck, not build (build needs the workflow-provided env vars).

Service routing (the proxy)

A global reverse proxy routes traffic by URL path to each artifact. The API server owns /api. Paths are NOT rewritten — services handle their full base path. For ad-hoc curl, always go through the proxy at localhost:80, never the service port directly.

```
localhost:80/api/healthz      !’ api-server (port 8080)   '  correct
```

```
localhost:8080/api/healthz    !’ bypasses proxy           '  wrong
```

4 · API Server — Deep Dive

Path: artifacts/api-server/src/. This is the largest and most important surface. Below is every meaningful file, what it does, and where to change things.

4.1 Server core & the middleware chain

src/index.ts	Process entrypoint. Calls verifyEnv(), initialises Stripe sync, starts HTTP server.
src/app.ts	Builds the Express app & middleware chain. Mounts Stripe + Paddle webhooks here (they need the RAW body for signature verification, so they sit BEFORE express.json).
src/routes/index.ts	Central router. Maps every sub-router to its base path (/auth, /tenants, /conversations, /widget, /billing, /ai, …).
src/lib/env.js	verifyEnv() hard-exits if DATABASE_URL, JWT_SECRET or GEMINI_API_KEY are missing. publicAppUrl() resolves the canonical public origin for links & redirects.

Middleware order (src/app.ts)

helmet !’ cors !’ pino-http (logging) !’ [raw-body webhook routes] !’ express.json !’ cookieParser !’ attachDb (injects the Postgres pool as req.db) !’ routes.

4.2 Authentication & JWT

File: src/controllers/auth.controller.js. Middleware: src/middleware/auth.js.

Endpoint	Lines	What it does
POST /auth/login	L82–112	Validate credentials !’ issue 15-min access JWT + 7-day refresh cookie (httpOnly)
POST /auth/refresh	L115–152	Rotate tokens using the refresh cookie
POST /auth/logout	—	Clear the refresh cookie
POST /auth/forgot-password	L165–223	Generate 1-hour reset token; send via SMTP or return in dev

How auth is enforced

- requireAuth — verifies the Bearer JWT with JWT_SECRET, attaches req.agent (id, tenantId, role, permissions).
- requireRole(...) — gate by role string: admin / agent / supervisor / superadmin.
- requirePermission(feature) — granular gating by feature key: inbox, billing, knowledge_base, etc.
- applyWorkspaceOverride — lets a superadmin "view as" a tenant by sending the X-Workspace-Id header.

JWT payload shape

```
{ sub: <agentId>, tenantId, role, permissions_json }
```

> **Access tokens live in memory, not localStorage**
> The dashboard keeps the access token in React state (XSS-resistant) and relies on the httpOnly refresh cookie for silent renewal. Never move the access token into localStorage.

4.3 Controllers & the full endpoint map

Each controller is an Express Router mounted at a base path in routes/index.ts. Change a feature by editing its controller.

Controller file	Base path	Owns
auth.controller.js	/auth	Login, refresh, logout, password reset

Controller file	Base path	Owns
signup.controller.js	/auth/signup	Public self-serve tenant signup
tenant.controller.js	/tenants	Workspace provisioning, brands, settings (SMTP/IMAP/ webhooks)
conversations.controller.js	/conversations	Inbox: list, messages, status, priority, ticket conversion
agents.controller.js	/agents	Invite agents, roles, set-password tokens
widget.controller.js	/widget	Visitor sessions, widget.js bundle, demo page, CSAT
super-admin.controller.js	/super-admin	Platform-wide: tenants, limits, plans, billing overrides
contacts.controller.js	/contacts	CRM view of visitors & history
canned-responses.controller.js	/canned-responses	Reusable reply snippets/shortcuts
email.webhook.controller.js	/webhooks/inbound-mail	Inbound email !' conversation threading
ai.router.js	/ai	Rephrase, summarise, knowledge-base crawl/upload, auto-reply
billing.router.js	/billing	Checkout, subscription state, plans

Key conversation endpoints (conversations.controller.js)

Endpoint	Lines	Notes
GET /conversations	L155–221	Paginated + filtered list (status, brand, agent, search)
PATCH /conversations/:id	L246–366	Update status/priority; converting to ticket sets is_ticket=true and moves to email channel
POST /conversations/:id/messages	L394–445	Agent sends message !' socket broadcast + email notify

4.4 Real-time engine (Socket.io)

File src/services/socket.service.js

Path /api/socket.io (NOT the default /socket.io — see gotchas)

Rooms

- tenant:{id} — inbox-wide updates for all of a tenant's agents.
- conv:{id} — everyone currently viewing one conversation.
- vis:{id} — a direct channel to a single visitor.

Key events

- client:send_message — visitor/agent sends a message.
- agent:is_typing / visitor:is_typing — typing indicators + collision detection.
- client:page_view / visitor:page_change — visitor telemetry (what URL they are on).
- visitor:online / visitor:offline — presence.
- server:new_message, conversation:created — server !' dashboard pushes.

AI auto-reply: for conversations in ai_handling status, the socket layer calls maybeAutoReply() which asks Gemini for a response.

4.5 AI layer

File	src/services/ai.service.js (Google Gemini)
Routes	src/routes/ai.router.js

- POST /ai/rephrase (L54–68) — agent copilot: rewrite a draft reply.
- POST /ai/knowledge-base/crawl (L216–270) — scrape a URL into knowledge articles.
- POST /ai/knowledge-base/upload-pdf — ingest a PDF into the KB.

RAG note: the ai_embeddings table (pgvector, VECTOR(1536)) is designed for semantic search but pgvector is NOT available on Replit Postgres, so that table is excluded from the applied schema. AI answers currently work without vector search.

4.6 Billing — dual provider (Stripe + Paddle)

Core file	src/lib/billingProvider.js
Controller	src/controllers/billing.controller.js
Plans repo	src/lib/plansRepo.js (billing_plans table = local source of truth)

How the provider is chosen

- New signups !’ the BILLING_PROVIDER env var (stripe default, or paddle).
- Existing customers !’ routed to whichever provider they already have (stripe_customer_id vs paddle_customer_id on the tenants row).

Checkout flow

1. Visitor picks a plan on the marketing site /checkout and enters details. 2. billing.controller.js !’ createPublicCheckout() calls the provider. 3. Stripe: Checkout Session with a 14-day trial in subscription_data. Paddle: a transaction built from a priceId (seeded by seed-paddle-products.ts). 4. User is redirected to the hosted checkout, then back to /checkout/success.

> **Paddle customer-conflict handling**
> When Paddle rejects "email conflicts with customer of id ctm_xxx", _paddleTenantCheckout extracts the existing customer id from the error message (falling back to a GET /customers?email lookup) and reuses it instead of failing. Keep this behaviour if you touch checkout.

Subscription state & the lock/grace mechanism

getSubscription() (billing.controller.js, L188–295) computes a lockState. When trial_ends_at passes, a grace period begins; after grace expires the tenant is locked. The dashboard renders TrialGateway.jsx as a blocking overlay. Superadmins can adjust trial_ends_at, grace_period_ends_at and reset lock_notified_at.

4.7 Webhooks

Webhook	Where	Purpose
Stripe	app.ts L69–100	stripe-replit-sync mirrors Stripe !’ local "stripe" schema; provisionTenantFromEvent reconciles tenant status
Paddle	app.ts L344–412	Verifies Paddle-Signature; public checkout provisions a tenant from scratch on purchase

Webhook	Where	Purpose
Inbound email	email.webhook.controller.js L249–437	POST /api/webhooks/inbound-mail — turns emails into conversations

Inbound email threading (email.webhook.controller.js)

- Normalises payloads from SendGrid, Resend, and Mailgun into one shape.
- Threads replies via reply+conv_<id>@ addresses (parseConvIdFromReplyTo) or the In-Reply-To header.
- stripQuotes (L50–56) removes quoted reply chains so only the new text is saved.

> **Why webhooks are mounted before express.json**
> Stripe and Paddle sign the RAW request body. If express.json parses it first, the signature check fails. That is why these routes use express.raw and are registered early in app.ts. Never move them below the JSON parser.

5 · Database — Schema, Tables & Relationships

PostgreSQL (Replit-managed). Baseline DDL is artifacts/api-server/schema.sql, applied MANUALLY (no Drizzle migrations). Several tables/columns are added at runtime by "self-healing" migrations that run when a controller/service boots.

5.1 The multi-tenant model

Logical isolation via tenant_id. The top-level entity is tenants; nearly every other row references it. A tenant has many brands; agents belong to a tenant and can be restricted to specific brands via the brand_access_array (a UUID[] — there is no join table).

5.2 Core tables (schema.sql)

Table	Key columns	Relations
tenants	id(PK), company_name, subscription_status, stripe_/paddle_/lemon_squeezy_ ids, grace_period_ends_at, created_at	root entity
brands	id(PK), tenant_id(FK), brand_name, widget_config_json, allowed_domains_array, inbound_email_prefix(UNIQUE), ai_system_prompt, ai_confidence_threshold	tenant 1-N
agents	id(PK), tenant_id(FK), name, email, password_hash, role, brand_access_array(UUID[]), personal_settings_json, is_active	tenant 1-N

Table	Key columns	Relations
visitors	id(PK), tenant_id(FK), brand_id(FK), session_token(UNIQUE), email, display_name, ip_address, location_city, last_seen_at	brand 1-N
conversations	id(PK), tenant_id, brand_id, visitor_id, assigned_agent_id, status, priority, csat_score, sla_breach_at, subject, channel	visitor 1-N
messages	id(PK), conversation_id(FK), sender_type, sender_id, message_body, is_internal_note, attachments_json, created_at	conversation 1-N
knowledge_articles	id(PK), tenant_id, brand_id, title, public_html_content, plain_text_content, is_public, author_agent_id	brand 1-N
password_reset_tokens	id(PK), agent_id(FK), token(UNIQUE), expires_at, used_at	agent 1-N
ai_embeddings	id(PK), article_id(FK), tenant_id, brand_id, embedding_vector VECTOR(1536) — EXCLUDED (no pgvector)	article 1-N

5.3 Runtime-created tables

These are created lazily by controller/service boot migrations — you will not find them in schema.sql:

Table	Created by	Purpose
billing_plans	lib/plansRepo.js	Local plan catalog (slug, amount_cents, limits, paddle_product/price_id)
super_admin_emails	super-admin.controller.js	Who has platform superadmin access
upgrade_requests	super-admin.controller.js	Tenant plan-upgrade requests (status: pending)
platform_settings	super-admin.controller.js	Singleton (id=1) — platform SMTP config JSON
canned_responses	canned-responses.controller.js	tenant_id, name, body, shortcut

5.4 Columns added to tenants at runtime

billing.controller.js and super-admin.controller.js ALTER the tenants table to add: account_status, plan, max_brands_allowed, max_agents_allowed, conversation_limit, ai_feature_enabled, smtp_feature_enabled, trial_ends_at, lock_notified_at, paddle_customer_id, paddle_subscription_id.

Schema quirks you must respect

visitors uses display_name (not name) — queries COALESCE(v.display_name, v.email, 'Visitor'). messages uses sender_id (not sender_agent_id). Both bit earlier developers; keep to these names.

6 · Dashboard Frontend — Deep Dive

Path: artifacts/dashboard/src/. React 19 + Vite + Tailwind v4 + TypeScript. Almost the entire app lives in a single large file, App.tsx (~5300 lines). Auth lives in context/AuthContext.jsx.

6.1 App.tsx component map

Use this table to jump straight to the feature you need. Line ranges are approximate — search the component name to confirm.

Section / Component	Lines	What it renders
Auth pages	630–866	Login, Signup, Forgot/Reset password
useApi (API client)	176–433	Every REST call as a method (listConversations, sendMessage, …)
ConversationsList + EmptyChat	867–1577	Inbox sidebar: filters, search, conversation rows
ChatPanel	1578–1881	Message history, internal notes, attachments, send box
Brands	1882–1979	Brand identities, widget config, embed snippet, domains
Billing	1980–2138	Plan selection, subscription state
CSAT	2139–2309	Satisfaction scores + agent performance
Settings	2310–2591	Profile + tenant-level config
Team	2592–2796	Agents, roles, permissions
SuperAdmin	2797–3681	Tenant provisioning, global billing, plans, limits
AI Training	3702–4112	Knowledge base editor + brand AI prompts
SMTP	4113–4273	Outbound mail server config
Tickets	4274–4415	Long-running tickets converted from chats
Contacts	4416–4692	Visitor CRM + history
Canned Responses	4693–4799	Snippet/shortcut manager
Sidebar (nav)	4800–4934	Global nav, unread counts, recent activity
Dashboard (main)	4935–5321	State orchestration, Socket.io lifecycle, section switch
Root App	5322–end	Auth gate + view routing

> **Where to change things**
> Want to edit the inbox? ConversationsList / ChatPanel. Billing UI? the Billing + SuperAdmin sections. A new API call? add a method to useApi (L176–433) then call it. Channel icons (incl. WhatsApp)? the ChannelIcon component. Navigation items? the items array in Sidebar.

6.2 The API client (useApi hook)

Rather than a separate library, App.tsx defines a useApi() hook (L176–433) returning an object of methods for every backend call. Each uses authFetch from AuthContext so requests are always authenticated. To add an endpoint: add a method here, then call it from the component.

6.3 Authentication (context/AuthContext.jsx)

Access token kept in memory (React state) — XSS-resistant.

- Refresh token in an httpOnly cookie; access token silently refreshed ~60s before expiry.
- authFetch — wrapper that injects the Authorization header and retries once on 401.
- Workspace override — superadmins inject X-Workspace-Id to act inside a tenant.
- can(feature, level) — frontend RBAC helper mirroring the server permissions.

> **AuthContext stays .jsx on purpose**
> It is JavaScript+JSX (not TSX). The dashboard tsconfig sets allowJs: true specifically for it. Do not rename it to .tsx without updating config and typing everything.

6.4 Real-time on the client

Socket connectivity is set up in the Dashboard component (~L5002–5107). It handles server:new_message, visitor:is_typing, visitor:page_change, visitor:online/offline, and conversation:created. Agents join a conversation room via join:conversation. Includes browser Notification support and an incoming-message chime.

6.5 Routing

State-based, not URL-based. A single section state (~L4948) decides which section component renders (~L5273–5297); the Sidebar calls setSection. Only minimal URL params are honoured (e.g. reset_token). Session persistence is via React state + the refresh cookie.

7 · Marketing Site & Embeddable Widget

7.1 Marketing site (artifacts/marketing-site)

Stack	React + TypeScript, Vite, Wouter routing, Tailwind, Radix UI, react-query
SSR	Custom Vite SSR; prerender.mjs renders routes to static HTML at build for SEO
Entry	main.tsx (client) + entry-server.tsx (SSR)
Layout	src/components/layout.tsx wraps every page (nav, footer, floating WhatsApp button)
SEO	src/lib/seo.ts + <Seo /> updates meta tags per route

Routes

- / home · /pricing · /checkout · /checkout/success
- /help + /help/* setup guides (Google Workspace, Microsoft 365, AI bot)
- /terms · /privacy · /refunds legal

7.2 Embeddable widget

Served by the API server as a self-contained IIFE at /api/widget/widget.js. Embedded on any customer site:

```
<script src="https://<your-domain>/api/widget/widget.js"
        data-brand-id="<BRAND_UUID>" defer></script>
```

- On load it starts/restores a session via POST /api/widget/session (returns/verifies a sessionToken UUID; dedupes visitors by email).
- Messaging over Socket.io (client:send_message) with a REST fallback POST /api/widget/message for attachments.
- If the tenant has AI auto-reply on, conversations start in ai_handling and Gemini answers via maybeAutoReply.
- On close, an optional 1–5 star CSAT survey posts to /api/widget/csat.

> **Widget CORS is intentionally wide open**
> widget.controller.js sets Access-Control-Allow-Origin: * and Cross-Origin-Resource-Policy: cross-origin (overriding Helmet) because the widget must load on ANY customer domain. The /api/widget/session endpoint is deliberately unauthenticated — it is visitor-facing. Keep widget responses shape-tolerant: the widget.js is cached on third-party sites and old versions stay in the wild.

8 · Environment Variables Reference

Manage these through Replit secrets — never hard-code them. The server hard-exits on boot if a REQUIRED var is missing (see lib/env.js).

Required (server will not start without these)

Variable	Used for
DATABASE_URL	PostgreSQL connection (also used by stripe-replit-sync)
JWT_SECRET	Signing/verifying access + refresh JWTs
GEMINI_API_KEY	Google Gemini AI (auto-reply, rephrase, summarise)

Feature / integration vars

Variable	Used for
BILLING_PROVIDER	stripe (default) or paddle — provider for new signups
PUBLIC_APP_URL	Canonical public origin for checkout redirects & email links (must match provider-approved domain)
PADDLE_API_KEY	Paddle API auth (or via Replit Connector)
PADDLE_ENVIRONMENT	sandbox or production
PADDLE_WEBHOOK_SECRET	Verifies inbound Paddle-Signature
PADDLE_<PLAN>_PRICE_ID	Per-plan Paddle price ids
Stripe (Replit integration)	Stripe keys provided via the Replit Stripe integration
R2_* / S3 creds	Cloudflare R2 / S3 for conversation export bundles
SMTP / platform mail	Outbound email (invites, password resets, notifications)

9 · Recipes — "Where Do I Change X?"

A quick lookup from feature !' files to edit. Follow the golden rule (carry tenant_id) on anything data-related.

I want to…	Edit these

I want to…	Edit these
Add a new REST endpoint	controllers/<x>.controller.js (route) !’ mount in routes/index.ts !’ add a useApi method in dashboard App.tsx !’ call it
Add a DB column/table	schema.sql (baseline) AND the boot migration in the owning controller; carry tenant_id; then read/write it in the controller
Change inbox behaviour	conversations.controller.js (server) + ConversationsList / ChatPanel in App.tsx (client)
Change billing / plans	lib/billingProvider.js, billing.controller.js, lib/plansRepo.js (+ seed-paddle-products.ts) + Billing/SuperAdmin sections
Adjust trial / lock logic	billing.controller.js getSubscription (lockState) + TrialGateway.jsx overlay + super-admin.controller.js overrides
Tune the AI	services/ai.service.js + per-brand ai_system_prompt (Brands / AI Training) + ai.router.js
Add a real-time event	services/socket.service.js (server) + Dashboard socket handlers in App.tsx (client)
Change the widget	widget.controller.js + the widget JS bundle; keep responses backward-compatible
Add a channel icon	ChannelIcon in App.tsx + the Channel union type
Add a marketing page	new component in marketing-site/src/pages + a <Route> in App.tsx + nav in layout.tsx

10 · Gotchas & Hard-Won Lessons

- Socket.io path is /api/socket.io (not the default /socket.io) so the path-prefix proxy routes it to port 8080. Both server and client must agree.
- Always restart the api-server after env var changes — JWT_SECRET must be present at boot or login throws.
- After adding a dashboard dependency, run pnpm --filter @workspace/dashboard install.
- pgvector is not available on Replit Postgres — the ai_embeddings table is excluded from the applied schema.
- visitors.display_name (not name) and messages.sender_id (not sender_agent_id). COALESCE display_name !’ email !’ 'Visitor'.
- Webhooks (Stripe/Paddle) must stay mounted before express.json — they need the raw body for signature verification.
- Controllers are CommonJS .js loaded via require(); the entrypoint/infra is .ts. Interop is handled by esbuild — do not convert wholesale to ESM.
- Static web artifacts publish with serve = "static"; a custom npx serve run command fails to open a port on publish.
- Widget.js is cached on third-party sites — keep widget endpoint responses shape-tolerant for old embedded versions.
- Verify apps with typecheck, not build — build needs workflow-provided PORT and BASE_PATH.

Demo credentials (development)
admin@omnicore.test / Admin123! · sara@omnicore.test / Agent123!